Advanced call scheduling for testing a telecommunications network

C Voudouris, N Azarmi and A Jones

This paper describes an advanced scheduling system and method for generating large volumes of calls to be used for testing a telecommunications network. The system is capable of preparing large-scale and complex network tests by viewing the task as a scheduling problem. The various requirements of the scheduling problem are analysed and represented as constraints or optimisation criteria. A fast heuristic method is proposed for solving the problem. The approach is based on a greedy algorithm for constructing solutions and it incorporates limited backtracking and dynamic value-ordering heuristics. The algorithm and system are currently being used for call charge verification in BT's PSTN and CSP networks.

1. Introduction

A telecommunications network requires frequent testing so that certain parameters of the system can be verified as being within the desired ranges. One approach for testing a large network is to have a set of test sites which automatically make calls one to the other while the network is in operation. These calls can be used for measuring various parameters of the system which otherwise would have been difficult or impossible to obtain from the live traffic.

One application of such a testing system is call charge verification, to ensure that no overcharging or undercharging of calls takes place. The test calls can be accurately measured and have their charge calculated. The calculated call charge for the test calls can then be compared with the actual charge for the same calls from BT's billing systems for the purpose of verification. Other potential applications for the system include call-quality measurements, testing of new telecommunications equipment and generally any type of test which requires sampling of data from the network's real traffic in the context of a controlled environment.

Generating and scheduling large volumes of calls for a large number of sites is a difficult task and requires satisfying a number of constraints, mainly due to the fact that sites are unary resources which can make or receive only one call at a time. Moreover, the minimisation of one or more criteria regarding the distribution of the calls is usually required. In this paper, the problem of scheduling calls for network testing is formally defined. The various requirements of the problem are analysed and represented as constraints or optimisation criteria. A heuristic search algorithm is proposed for solving the problem.

2. Network testing architecture

An on-line testing architecture for a network such as the PSTN usually includes a number of test sites across the network which are representative, with regard to their geographical distribution, of the actual network topology. Each of these sites is equipped with a microcomputer system and a modem which can be remotely programmed to make calls at precise times to the other sites of the testing architecture [1].

This facilitates the automated sampling of a certain behaviour of the whole network while the network is in operation. Conclusions can be safely drawn if the tests are properly set up to reflect as closely as possible the call patterns emerging in the network in its day-by-day operations.

The centre of the architecture is the master controller where the tests are set up and monitored. This point of

control is equipped with an array of modems so that it can contact the test sites. The main role of the master controller is to feed the test sites with instructions, monitor their status, collect the results from the various tests, and finally pass these results to other systems for analysis. There may be more than one master controller. This depends on the total number of sites that are part of the network testing system and also the maximum number of sites that can be managed by a single master controller. In some cases, there are also back-up controllers which can take over if there is a failure in the main controllers.

The instructions passed by the master controller to the test sites are mainly regarding the calls that should be made by these sites to execute a test. In particular, each test site is contacted once a day and the schedule data is uploaded which instructs the test site what calls to make, where and when in the next 24 hours. The time when the master controller normally contacts a test site is called the update time of that site.

All sites do not have the same update times because of the limited number of modems on the master controller side which can contact only a limited group of sites at a time.

In total, there are three different types of site:

- outgoing/incoming (both ways) sites — these sites can both make and receive calls (most of the sites in the system are usually of this type),
- outgoing sites — these sites are only capable of making calls,
- incoming sites — these sites are only capable of receiving calls.

The network is divided into a number of geographical areas (called zones for the purposes of describing the system) with the various test sites being distributed between them. Usually calls are made between sites in the same zone but there may also be test calls across zones. The system is capable of simulating different types of calls such as chargeable, non-chargeable (i.e. calls, to destinations which are busy), chargecard calls, etc.

To provide the sites with schedules, the master controller is communicating with a call generation and scheduling component which determines the call pattern to be run for the test. In the context of the Call Charge Verification by DETECT (CCVD), the project which is the drive behind this work, this component is called the Sequencer and it is the point of contact between the user who wants to run a test and the testing architecture.

The main role of the Sequencer is to prepare the schedules for all the test sites by solving the call scheduling problem to be explained in the next section. The user of the Sequencer is assisted by a user interface in defining the problem (or the test from their point of view). Once the problem is defined by the user, the core algorithm of the Sequencer is called which solves the problem and produces the exact schedule data for the test sites. This schedule data is subsequently passed to the master controller for distribution to the test sites. Normally, tests are defined on a weekly basis which means that the sequencer has to schedule calls for all the sites and for a total of seven days in a single session. Figure 1 shows an example network testing architecture as described thus far. For more information on testing architectures, the reader may refer to a patent by BT in this area [1].

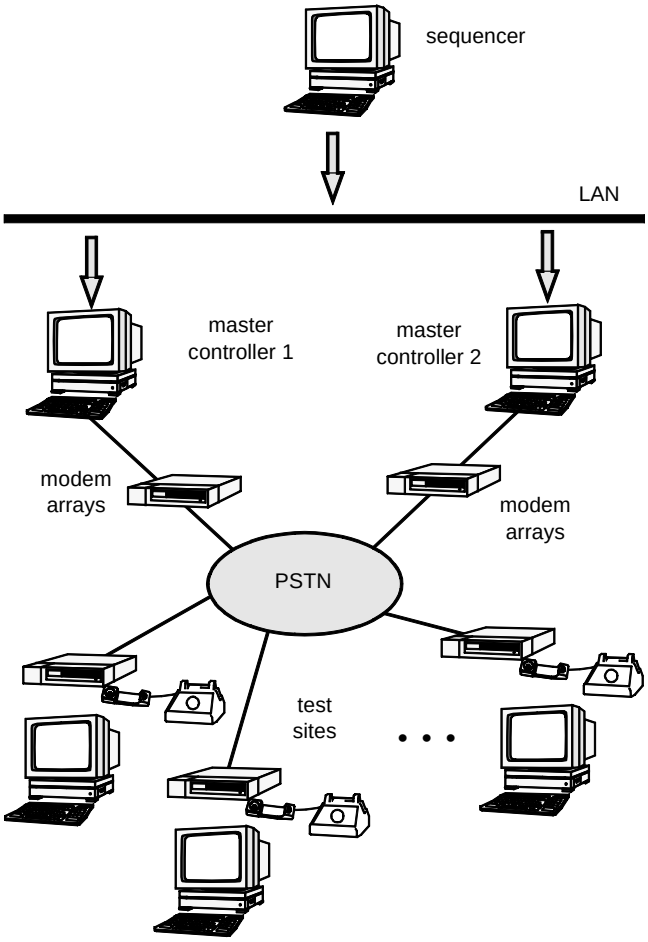

Fig 1 An example of a network testing architecture.

3. Call scheduling for network testing

In general, a system for generating test calls must ensure that during the time that a call is to be received by a site, this site would not be instructed to make another call. In addition to that, for certain call types, the system must also

ensure that the destination of the call will be available to receive the call.

Essentially, test sites are unary resources from a scheduling perspective, allowing only one operation to be performed (i.e. receive or make call) at a time. Each call can be seen as an activity requiring one or more resources (i.e. sites) to be allocated to it.

For a normal call, there are two resources to be allocated which are the source site for the call and the destination site for the call.

For other types of call such as three-party calls, more than two sites might need to be allocated, while for non-chargeable calls (calls to a busy destination) only one site need be allocated (i.e. the source site), but it must be ensured that the destination site to be selected, although not allocated, is busy at that time by another test call.

During the set-up of the testing architecture for BT within the framework of the 'DETECT' project, the difficulty of the problem was not initially obvious as it was not identified as being of a combinatorial search nature. In the first experiments, a simplistic approach was adopted where a group of sites was calling another group of sites during half of the day and the opposite was happening during the other half. Although this simple scheme satisfied the problem's constraints since there were no conflicts between the calls, it was of limited use with regard to being representative of the live network traffic. In particular, the pattern of calls emerging was highly structured and in no way resembled the actual operation of the network.

Engineers trying to insert some randomness with regard to the call time soon realised that the unary resources (i.e. sites) were making it very difficult to find a conflict-free call pattern.

The idea of scheduling the calls using constraint technology then emerged, which resulted in the approach presented in this paper which realises a heuristic search method capable of generating conflict-free call schedules for tens of thousands of calls and hundreds of sites in reasonable time. The next sections describe the problem and explain the approach adopted.

4. Defining the problem

Constraint satisfaction problem (CSP) is the problem of finding an assignment of values to a number of discrete variables with finite domains such that a set of constraints is satisfied [2]. Assignments that satisfy the constraints are called solutions of the CSP. If the goal is also to find the solution that optimises a cost criterion then this variation of the CSP is called a constraint satisfaction optimisation problem (CSOP) [2]. The call scheduling problem could be classified as a CSOP. Intuition suggests that the call scheduling problem is NP-hard which means that no algorithm exists which finds the optimal solution in polynomial number of steps [3]. It may be difficult though to produce a formal proof for that.

The following provides a formal description of the call scheduling problem by explaining the entities and their relationships.

4.1 Problem instance

An instance of the call scheduling problem can be fully defined by the following list of sets and functions. These sets and functions are not directly provided by the user but they are generated by the system after pre-processing of both the user input and data describing the network testing architecture.

- Sets

  — Set of zones (Z) — $Z = \{z_1, z_2, ..., z_{ip}\}$ where I is the number of zones.

  The test sites are divided into a number of zones which correspond to big geographical areas such as Midlands, Scotland, etc.

  — Set of call types (Ct) — $Ct = \{ct_1, ct_2, ..., ct_J\}$ where J is the number of call types.

  Depending on the testing required there may be one or more call types (e.g. chargeable, non-chargeable, FeatureNet call, chargecard call, etc).

  — Set of charge bands (Cb) — $Cb = \{cb_1, cb_2, ..., cb_K\}$ where K is the number of charge bands.

  Usually, three charge bands (local, regional, national) are considered, but others, such as free-call, mobile, etc, may also be considered.

  — Set of tariff rates (Tr) — $Tr = \{tr_1, tr_2, ..., tr_L\}$ where L is the number of tariff rates.

  Examples of tariff rates are the day rate, the night rate, and the weekend rate. This set gives all the possible tariff rates that may apply in the network to be tested.

  — Set of sites (S) — $S = \{s_1, s_2, ..., s_M\}$ where M is the number of sites.

This set gives all the sites that are available in the network architecture for making or receiving calls

— Set of site types (*Tp)* — $Tp = \{I, O, B\}$.

As mentioned above there are three different types of sites which are incoming, outgoing, and both ways (i.e. both incoming and outgoing calls capability).

— Set of calls (C) — $C = \{c_1, c_2, ..., c_N\}$ where N is the number of calls.

These are the calls that need to be scheduled. The number of calls and their attributes are specified by the user with the assistance of the user interface.

— Set of time-slots (T) — $T = \{t_1, t_2, ..., t_o\}$ where o is the number of time-slots in the scheduling horizon (i.e. time period during which the calls are to be generated).

This set controls the granularity of the scheduling system. Each call can be allocated one or more consecutive slots from this set. The actual duration of the calls is specified by the user in terms of milliseconds. The scheduling system ensures that the total duration of the consecutive slots allocated to a call is equal to or greater than the duration in milliseconds of that call. Usually, 1-minute slots are of sufficient accuracy for the purposes of efficient scheduling and also for not wasting too much time in not fully utilised slots. If the scheduling horizon is 24 hours, this results in a total of 1440 slots for allocation for each site. T in this case is equal to { 1, ... , 1440 }.

- Functions

The sets listed above give an accurate picture of the different entities of the problem. The different relations between these entities can be defined by the following list of functions.

— Call information function (CIF):

$(z, ct, cb, tr) = CIF(c), C \rightarrow Z \times Ct \times Cb \times Tr$

This functions gives, for each call c that requires scheduling, the values for the various call attributes. Attributes related to the scheduling process described in here are the zone of the call z, the type of the call ct, the charge band of the call cb and its tariff rate tr.

— Site information function (SIF):

$(z, tp) = SIF(s), S \rightarrow Z \times Tp$

For each site s, it is required to know its zone z and type tp. The above user-defined function is providing that.

— Site availability function (SAF):

Available = $SAF(s, t), S \times T \rightarrow \{0, 1\}$

This function gives us the availability of a site s during the time slot t. Using this function, the user may specify maintenance periods, update times for the sites, etc. The algorithm has to ensure that no calls are scheduled during these periods.

- Incoming site function (ICS):

— Destinations = $ICS(s, cb), S \times Cb \rightarrow 2^S$

This function takes as arguments a site s and a charge band cb and returns a list of other sites in the testing architecture for which a call made from site s to them would belong in the charge band cb.

— Tariff rate function (TR):

$tr = TR(t), T \rightarrow Tr$

This function gives the tariff rate for a certain time-slot of the scheduling horizon.

— Endtime function (ET):

$et = ET(st, cb, ct), T \times Cb \times Ct \rightarrow T$

The user has also to specify the required duration of calls. This function provides this facility by giving the end time-slot et for a call starting at the time-slot st, with charge band cb and call type ct. The call will take place during all the consecutive slots starting from st up to et. As a matter of convention the slot with the value et is excluded.

All the sets and functions mentioned so far can fully specify an instance of the call scheduling problem. The algorithm has to generate a solution to this problem instance which is of the following form.

4.2 Solution

A solution to the problem is a scheduling function of the form:

$$F: c \in C \rightarrow (s_o , s_i , s_t) \in S \times S \times T$$

This means that for each call requiring scheduling, the problem solver has to specify three decision variables. These are the origin of the call s_o (outgoing site), the destination of the call s_i (incoming site) and the start time slot s_t. This solution should satisfy the following constraints.

4.3 Constraints

Let us consider the following definitions for a single call c:

$(z, ct, cb, tr) = CIF(c)$,
$(z_o, tp_o) = SIF(s_o)$,
$(z_i, tp_i) = SIF(s_i)$,
$et = ET(s_o, st, cb, ct)$,

For the solution to be feasible, the following conditions should hold.

- $z_o = z$

  The zone of the source site z_o should be equal to the zone z of the call.

- $tp_o = O \vee tp_o = B$

  The type of the source site should be either outgoing or both ways so that it can make the call.

- $tp_i = I \vee tp_i = B$

  The type of the destination site should be either incoming or both ways so that it can receive the call.

- $s_i \in ICS(s_o, \text{cb})$

  The destination site s_i should belong to the set of sites for which a call from s_o will be in the desired charge band cb.

- $tr = TR(st)$

  The tariff rate for the start time st of the call c is equal to the desired tariff rate tr for the call.

Additionally, let us consider the time slots $T'c$ during which the call c is going to take place:

$$T'c = \{t \in T \mid st \leq t < et\}$$

During this time the tariff rate should remain the same and both the source and destination sites should be available (i.e. no update times, maintenance periods). This can be expressed as follows:

$$\forall t \in T'_c,\ TR(t) = tr_o,\ SAF(s_o, t) = 1,\ SAF(s_i, t) = 1,$$

Finally, the resource constraints are dictating that all sites should perform only one operation at a time (i.e. either receive or make calls). This can be expressed as follows.

$$\forall c_i, c_j \in T,\ c_i \neq c_j\text{: if } (T'_{ci} \cap T'_{cj}) \neq \varnothing$$

$$\Rightarrow (\{s_o(c_i), s_i(c_i)\} \cap \{s_o(c_j), s_i(c_j)\} = \varnothing)$$

The above formula may take other less strict forms in certain network testing problems. For example, if the requirement is to test non-chargeable calls (where the destination has to be engaged so that the customer is not charged), the source of a normal chargeable test call and the destination of the non-chargeable test call must be set to the same value.

4.4 Optimisation

Leaving behind the various constraints of the problem, there are various criteria that the solution has to optimise. There are two main objectives for this problem.

- Objective A — spread calls across sites in the same zone

  This objective states that the utilisation of sites in the same zone should be as uniform as possible.

- Objective B — spread calls throughout the day

  This objective states that the calls for a site should be as uniformly distributed as possible with respect to their start times.

5. Solving the problem

The problem does not require any sequencing of calls nor imposes any hard or soft constraints with regard to the start times/end times of calls. Therefore, from a scheduling point of view, it is not a very difficult problem.

The real difficulty of the problem comes from the fact that realistic testing scenarios (at least in the case of BT's PSTN network) may require up to 350 000 or more calls to be scheduled in a single session. This may result in a total of 1000 000 or more decision variables to be specified in a relatively short time. It seems that real-world instances of the call scheduling problem (as attempted here) are among the largest scheduling problems as far as the number of decision variables is concerned.

Faced with a search space of that size, exhaustive search is not a viable option. For that reason, development was directed towards a heuristic 'solution generation' (greedy algorithm) approach.

The proposed algorithm is receiving the list of the calls to be scheduled and then starts to schedule them one by one. If a call cannot be scheduled then the algorithm does not backtrack to try different assignments for the calls already scheduled but reports the problem back to the user. Given that the tests do not require full utilisation of all the network sites, the case of failing to schedule a call is very rare and if it happens means that the proposed test case is very close to the limits of the test sites with respect to the total number of calls that they can make.

The following looks more closely to the algorithm for allocating a single call which is the core of the system since

it is simply repeated for all the calls that need to be scheduled (until either all of them have been allocated or one or more calls have failed the allocation process).

For each site in the system, a timeline is defined. This timeline gives, for the time-slots defined over the scheduling horizon, the availability of the site during these times. Before allocating any calls, all the sites' timelines are initialised with the update times of sites and also any other system maintenance times that may be required.

When allocating a single call, the timelines of all the possible sites that can be sources or destinations for the call are being searched, in an attempt to locate a free set of slots which can accommodate the call.

Once a decision is made regarding the source and destination of a call, the timelines of the respective sites are updated and the solution generation proceeds with allocating the next call in a static order.

The algorithm for allocating a single call is the following:

1 a list is created which contains all the possible outgoing or both-way sites in the zone of the call,

2 sites in this list are sorted by their utilisation in ascending order (this value-ordering heuristic pursues Objective A — moreover it reduces the time required to find a gap in the timelines to schedule the call),

3 select the first site in this list to be the origin of the call, then remove this site from the list — if all the sites have been tried and failed (list is empty) then go to step 8,

4 for the site selected in step 3, choose a random time-slot as a starting point (this is the heuristic which enables calls to be spread uniformly throughout the day),

5 starting from the time-slot currently selected, find the first big enough gap (i.e. set of available slots) in the timeline which can accommodate the call in such a way that all other constraints are also satisfied (i.e. appropriate tariff rate, no tariff rate change during the call) — if a full cycle on the timeline is completed from the random starting point selected in step 4, go to step 2 and try another site as the source site,

6 find destinations for the call's charge band and order them by utilisation in ascending order,

7 try the destinations in this order to find one that is available for the duration of the call — if no such destination exists, go to step 5 and continue searching the timeline; if a destination exists, schedule the call and exit,

8 advise user about failure to schedule call and provide information about the network area currently overbooked.

Readers interested in a more elaborate description of the method, may refer to the patent by BT for the sequencer system [4].

Experiments have been carried out to evaluate the performance of the overall algorithm. These experiments included both randomly generated problems and real test data. Table 1 gives the times required by the algorithm for solving randomly generated problems of varying size and difficulty. Table 2 shows the results obtained for real test data in one of the zones of the 'Call Charge Verification using DETECT' system. All experiments were carried out on a Sun SPARCstation 10 machine.

A sample call schedule for a single day (24 hour period) in the form of a Gantt chart, as taken from the Sequencer's schedule visualiser, is shown in Fig 2. As can be seen, a large volume of calls is generated by the system. The pattern is more dense during daylight (centre of the Gantt chart) while becoming less dense during the early morning and late night hours (left and right end of the Gantt chart). The emergence of patterns of gaps can also be seen in the left hand side. These correspond to the update times of the test sites during which no calls are being scheduled.

Table 1 Results for a set of randomly generated problems.

No. zones	No. sites	No. calls	CPU time in seconds	Mean site utilisation	Min site utilisation	Max site utilisation
10	700	50 000	103.11	10.95%	10.69%	11.31%
10	700	100 000	171.85	17.86%	17.43%	18.05%
10	700	150 000	223.09	24.65%	24.51%	25.13%
10	700	200 000	302.54	31.55%	31.25%	31.87%
10	700	300 000	463.82	45.25%	44.93%	45.55%
10	700	400 000	660.59	58.94%	58.75%	59.63%
10	700	500 000	794.01	72.63%	72.63%	72.63%
10	700	555 000	919.03	79.42%	79.16%	79.79%

Table 2 Results for the zone North Home Counties which contains 75 sites.

Day of the week	No. calls	CPU time in seconds (up to that day)	Mean site utilisation	Min site utilisation	Max site utilisation
Sun	6000	74.07	88.23%	82.08%	89.16%
Mon	5400	102.88	61.18%	55.48%	62.63%
Tue	7800	154.9	73.06%	70.27%	73.68%
Wed	7800	207.1	73.06%	70.27%	73.68%
Thu	7800	259.09	73.06%	70.27%	73.68%
Fri	4800	283.33	52.77%	49.65%	53.33%
Sat	6000	359.03	88.23%	82.08%	89.16%

6. Summary and conclusions

In this paper, we presented a scheduling approach to test call generation for network testing. The problem of call scheduling for network testing was formulated and a heuristic solution generation algorithm was described which is capable of creating very large and complex call patterns in reasonable time. The Sequencer system based on the principles described in the paper has been in use since July 1996 for the PSTN network and more recently for the CSP

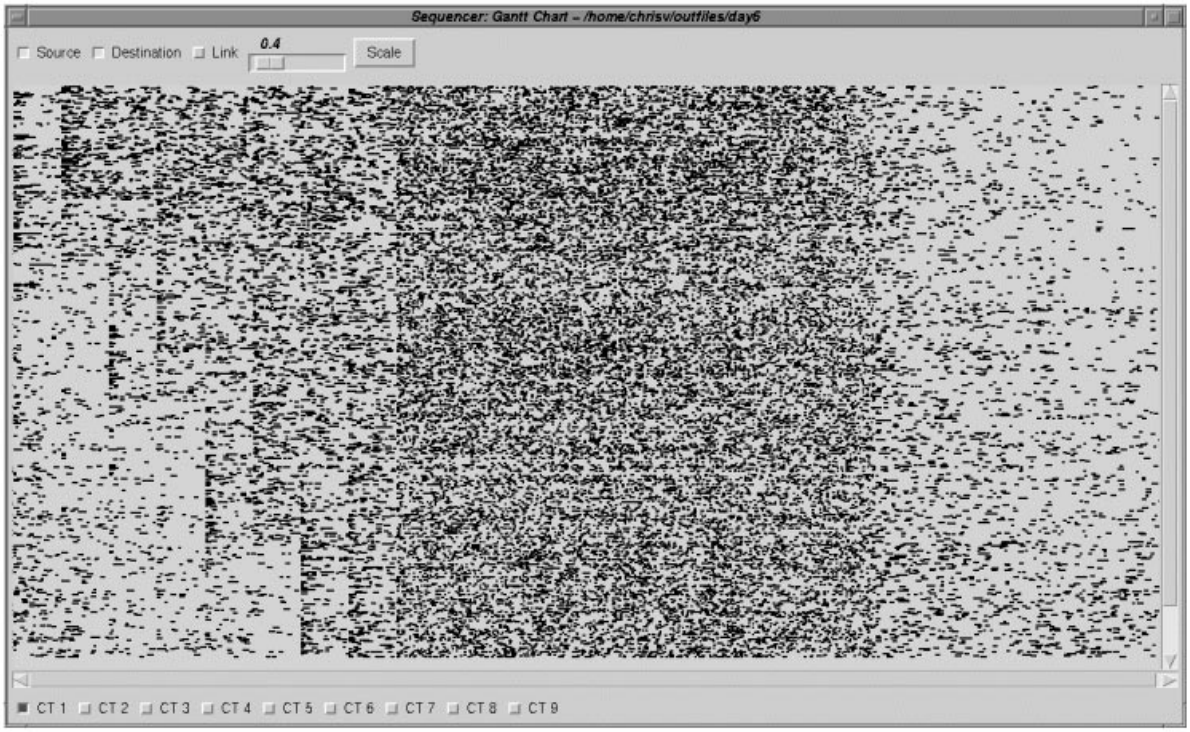

Fig 2 A sample call schedule visualised as a Gantt chart.

network as part of the Call Charge Verification using DETECT system. Current work is focused in extending the algorithms to facilitate testing of other BT networks such as FeatureNet and Cashless. Future research will focus on scheduling more complex types of network services which are not covered by the current version. Dynamic scheduling is also envisaged in the context of a more flexible testing environment that will allow schedules to be changed quickly while they are already in use.

References

1 Patent filed by BT on: 'Testing telecommunications equipment', International Application No. PCT/GB94/01557.

2 Tsang E: 'Foundations on constraint satisfaction', Academic Press (1993).

3 Garey M R and Johnson D S: 'Computers and intractability: a guide to the theory of NP-completeness', W H Freeman, San Francisco (1979).

4 Patent filed by BT on: 'A method for scheduling calls', GB Application No: 9714227.7.

Christos Voudouris holds a Diploma degree in Electrical and Computer Engineering from the National Technical University of Athens (1992), an MSc in Intelligent Knowledge Based Systems from the University of Essex (1993), and a PhD in Computer Science from the University of Essex (1997). During the years 1994—1995, he worked as a senior research officer in the Department of Computer Science at the University of Essex conducting research on heuristic search techniques for combinatorial optimisation problems. He joined the Intelligent Systems Research group at BT Laboratories in October 1995. Since then he has been working on research projects in the areas of resource scheduling, intelligent agents, and machine intelligence. He currently leads a project on the use of constraint optimisation techniques to resource scheduling applications. His research interests are combinatorial optimisation, heuristic search techniques, dynamic scheduling, intelligent agents, and fuzzy logic.

Nader Azarmi manages the Intelligent Systems Research group at BT Laboratories. He joined BT in December 1989, working on the application of AI techniques to telecommunications network management.

He went on to set up a research and development programme in the application of AI planning and scheduling technology to BT's resource management problems.

He is currently managing an international AI and Machine Intelligence research programme. He received a BSc (Hons) in Computer Science and Mathematics from the University of Manchester, an MSc in Artificial Intelligence and a MPhil in Computing Science from Essex university.

He is currently completing a PhD in Computer Science at Imperial College, London University.

Alan Jones joined BT in August 1972 as an apprentice. He split his time between exchange construction, installing and extending telephone exchanges, and customer apparatus and line repair. Following a spell of exchange maintenance, where he concentrated on network fault identification, he was re-deployed in 1988 to BT Laboratories.

Since joining BT Laboratories he has put his network expertise to good use in developing distributed testing techniques, now known as DETECT.

This allows testing devices installed at various points within a geographically remote system under test, to be controlled remotely by a centralised controller. This has found application in the area of customer metering approvals and he continues to develop the system for this purpose.